Week 3: Trace Processing and Workload Characterization

Oct 11, 2026

Week 3 turned TraceLab's raw logs into a pause dataset for our cost model, and described what those pauses look like.

Summary

Every pause in the TraceLab trace is now one row in a shared table (351,497 usable pauses), and five findings shape the rest of the project.

- **Tool pauses are mostly short; human pauses are mostly long.** At a 10 s break-even, 89% of tool pauses end in time (keep the cache), but only 12% of human pauses do (evict).
- **That split is known when the pause starts.** If the model just called a tool, it is a tool pause; if not, the agent is waiting on a person.
- **Within tool pauses, the tool name alone predicts little.** Deciding by tool name is 89.8% correct versus 89.3% for always keeping, and catches only 6% of long pauses.
- **Two tools carry most of the hard cases.** Bash and write_stdin are 37% of tool pauses but 79% of the tool pauses longer than 10 s.
- **The break-even time differs per pause.** Idle caches have a median of about 125K tokens, so b spans about 4–26 s at an assumed prefill speed of 10,000 tokens/s.

What changed in version 2

Hangting found that version 1 mismeasured about 32,000 Codex pauses; version 2 fixes them, and every number in this report comes from version 2.

- **The problem.** Newer Codex logs sometimes record a round's own tool results inside that same round, and write `usage_report` after the tools finish. Version 1 assumed results always land in the next round, so for those pauses it counted the model's next stretch of thinking as idle time and credited the pause with a tool called later.
- **The fix.** Each pause is now credited with the tools the model called in that round. It starts at the model's last output and ends when those tools' results are back, or when the person's message arrives.

- **The effect.** Codex tool pauses now match the tools' own reported runtimes 98% of the time, up from 82%. The Codex tool-pause median falls from 1.05 s to 0.33 s, and usable pauses rise from 346,820 to 351,497.

Data

We use the public TraceLab trace, release v0.0.1 (repo, paper): real Claude Code and Codex sessions recorded from September 2025 to June 2026.

Measure	Value
Sessions	4,265
Users (pseudonymous)	43
LLM rounds	357,161 (Claude 140,338, Codex 216,823)
Tool calls	432,510
File	`syfi_coding_trace.duckdb`, about 100 MB

We read three of its tables: `rounds` (one row per LLM call), `tool_calls`, and `timing_events` (one row per timestamped input or output inside a round).

Defining a pause

A pause runs from the moment the model finishes generating in round k to the moment the next LLM call has everything it needs. During that span the session's KV cache sits idle, so its length is T in the cost model.

- **Start:** the model's last output in round k (`reasoning`, `text` or `tool_call`). Codex's `usage_report` also counts, but only when it comes before the round's first tool result; in newer Codex logs it is written after the tools finish.
- **End:** when the last result of the tools round k called comes back, or, for a human wait, when the person's message that the next call reads arrives.
- **Tools:** the tools the model called in round k. They are known the moment the pause starts, so they are safe to use for prediction.
- **Older Codex logs** (106 sessions) write each tool call together with its result when the tool finishes, so the call looks late on paper. Those sessions keep the version 1 rule, which matches the tools' own runtimes there.

Each pause gets a cause from what the next LLM call was waiting for.

Cause	Meaning	Pauses
tool	only tool results	315,580
user	only a person's message: the agent finished and waited	33,411
tool+user	both	3,573
end	the session ended, so there is no next round	4,265
unknown	no tool was called and no message arrived	332

We checked the rule against each tool's own recorded runtime (agreement = within 1 s or 20%). Tool pauses agree 99% of the time for Claude and 98% for Codex; in the older Codex logs, single-tool pauses agree 89% of the time. No pause is credited with a tool called after the pause started.

The pause table

`pauses.parquet` holds one row per round (357,161 rows); analyses should keep only rows where `usable` is true.

Column	Meaning	Known when the pause starts?
provider, user, session_id, round_index, model	identifiers, used for train/test splits	yes
pause_s	T: seconds the cache sat idle	no, it is the target
cause	tool, user, tool+user, end or unknown	partly: n_tools = 0 means a user wait
usable	true for rows we trust	used to filter rows
context_tokens	tokens in the cache (input plus output of round k); drives B	yes
n_tools	number of tools the model called in round k	yes

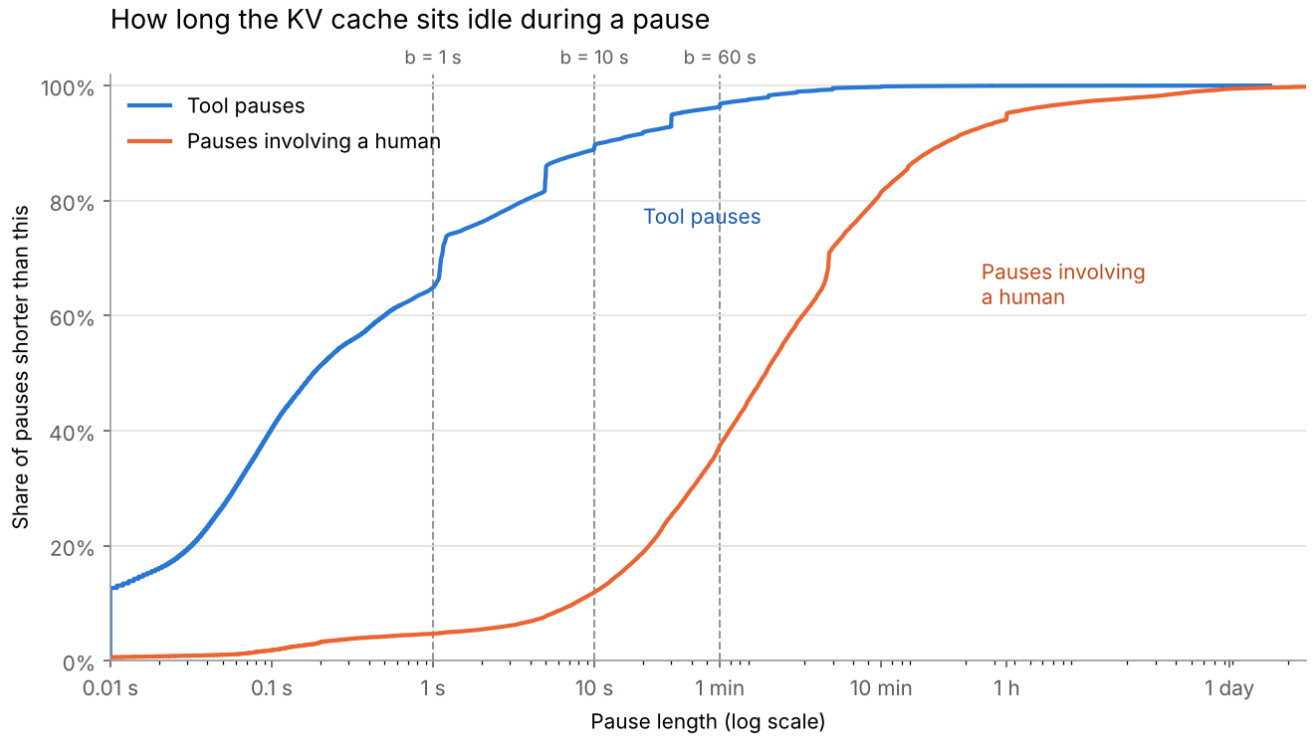
Column	Meaning	Known when the pause starts?
tool_names	their names, comma-joined	yes
tool_input_chars	total size of their inputs	yes
pause_start_at, pause_end_at	when the pause started and ended, for replaying sessions in order	start yes, end no
log_format	claude, codex or codex_old	yes

Excluded rows. 5,664 rows have `usable = false`: session ends (4,265), unknown causes (332), 976 rows next to repeated round numbers, 60 pauses whose tools never logged a result, and 31 where the person's message arrived while the model was still writing.

Left out on purpose. Result size, error flags and tool latency are known only after the pause ends, so a predictor that used them would be cheating.

Finding 1: tool pauses mostly beat the break-even time; human pauses mostly don't

At b = 10 s, keeping the cache wins for 89% of tool pauses but only 12% of pauses involving a human.

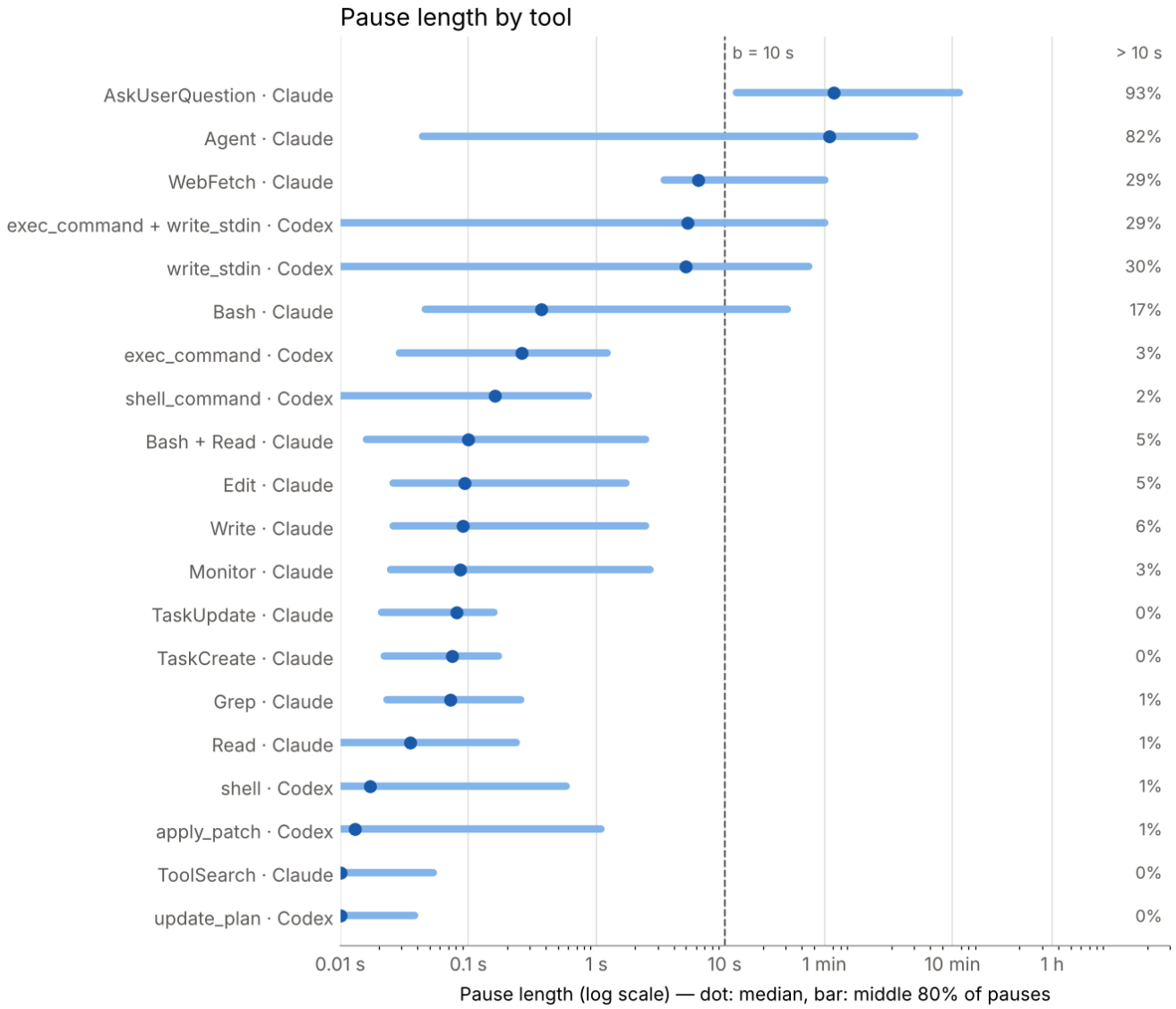


Break-even b	Tool pauses shorter than b	Human pauses shorter than b	All pauses
1 s	64.9%	4.7%	58.5%
5 s	84.7%	7.7%	76.6%
10 s	89.2%	11.9%	81.1%
30 s	94.1%	25.3%	86.9%
60 s	96.6%	37.4%	90.4%

The sharp jumps at 1 s and 5 s are Codex `exec_command` and `write_stdin` calls that wait a fixed time before returning partial output. Those pauses are very predictable.

Finding 2: the tool name alone predicts little

On held-out sessions, deciding keep or evict at b = 10 s by tool name alone is 89.8% correct, barely above always keeping (89.3%). It catches only 6.4% of the long pauses.

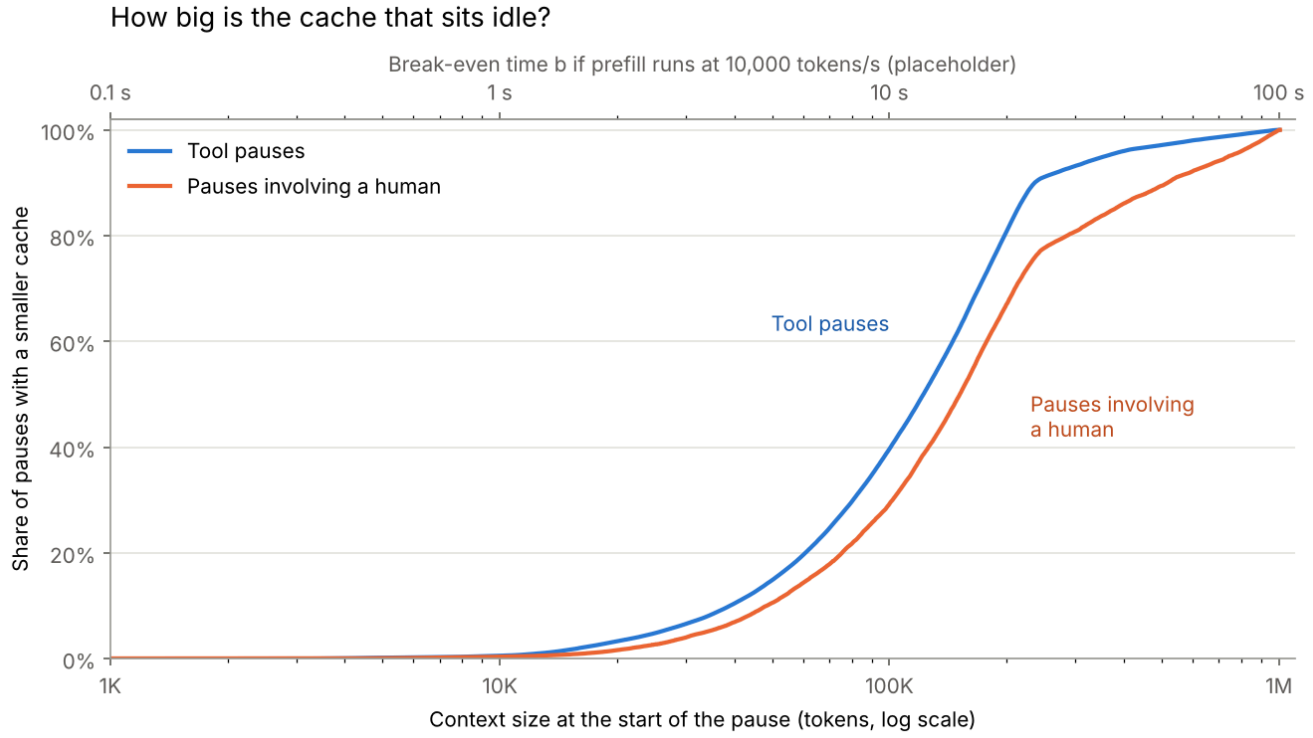


- **Easy calls.** Read, Grep, Edit, exec_command and apply_patch rarely pass 10 s (1–5%), so keep; AskUserQuestion (93%) and Agent (82%) almost always do, so evict.
- **Hard calls.** Bash (17% over 10 s) and write_stdin (30%) straddle b. Together they are 37% of tool pauses but 79% of the tool pauses longer than 10 s.
- **Hidden human waits.** AskUserQuestion is a person answering, so it behaves like a human wait.

Version 1 also listed exec_command and apply_patch as hard; that came from the mismeasured Codex pauses. The check learns each tool's majority answer on 80% of sessions and tests on the other 20%.

Finding 3: idle caches are large, so b differs per pause

The median pause leaves about 125K tokens in the cache, so b spans about 4–26 s (10th to 90th percentile) at 10,000 tokens/s.



Claude's human waits carry the largest caches: a median of 213K tokens and a 90th percentile of 740K. Codex stays near 121–126K. For scale, a Llama-3-8B-class model stores about 128 KB of cache per token, so 125K tokens is about 16 GB.

Giving each pause its own b = context_tokens ÷ prefill speed (queueing delay 0) keeps the picture from Finding 1:

Prefill speed (placeholder)	Median b	Keep wins: tool pauses	Keep wins: human pauses	Keep wins: all
5,000 tokens/s	25.0 s	92.9%	24.7%	85.7%
10,000 tokens/s	12.5 s	89.4%	16.3%	81.7%
20,000 tokens/s	6.3 s	84.7%	10.7%	76.9%

The speed moves these shares by several points, so final results need M5's measured speed or a sweep across this range.

What Week 3 means for the project

Our research question holds up on real data: the keep-or-evict decision matters, predictions will be imperfect, and that is exactly the case our policy is built for.

1. **There is room for a smart policy.** At b = 10 s, keeping is right for 81% of pauses and evicting for the other 19%, so neither always-keep nor always-evict is optimal.
2. **Part of the prediction is free.** Whether the agent called a tool or is waiting on a person is known when the pause starts, and it separates mostly short pauses from mostly long ones.
3. **The rest is genuinely hard.** Inside tool pauses, the tool name barely beats always keeping, so even a good predictor will make mistakes. The safety window (λ) is designed for this: trust predictions when they are right, and limit the damage when they are wrong.
4. **The break-even time must be computed per pause.** It scales with each pause's context size, and the prefill speed behind it is the main number we still need to measure.

Next, Week 4 builds the duration predictor from this table, and the tool-name rule from Finding 2 is the baseline it has to beat.

What this means for each workstream

Every workstream can start from `pauses.parquet` now; each has one or two specific changes.

Workstream	What to take from Week 3
M1 Algorithm	Use a per-pause b = prefill(context_tokens) + queueing delay. Session ends (4,265) never return, so OPT = 0 there; the safety window caps that loss at B/λ per session end.
M2 Workloads	The pause table and three figures are the Week 3 deliverable. Replace the proposal's 38 s / 44 min figures: they are TraceLab's whole-request response times, not single pauses.

Workstream	What to take from Week 3
M3 Prediction	n_tools separates tool from human pauses for free; the real work is the tail of Bash and write_stdin. Add input size and recent pauses in the session, and consider the command details the JSONL release keeps.
M3 splits	Split by session. Claude versus Codex is a ready-made distribution shift for C4.
M4 Simulation	Drive the simulator from `pauses.parquet` and sweep prefill speed. TraceLab's `eviction_tradeoff` experiment already sweeps a fixed timeout, a ready-made fixed-TTL check.
M5 Systems	Measure prefill time against context length on the GPU we plan to use; it moves the keep-wins shares by several points.

For the novelty map: TraceLab's related work cites CacheWise, which builds predictive eviction from tool-call metadata. It is close to C1 and C3, so it belongs on the literature map.

Limitations and open questions

The pause measurements are solid; the break-even numbers rest on two placeholders.

- **Prefill speed is assumed** (5,000–20,000 tokens/s, linear in context). Real prefill grows faster than linearly at long contexts.
- **Queueing delay is set to 0.** Under load, b grows, so the simulator should model it.
- **Some user messages may be automatic.** Permission replies or injected notes can be logged as `user_message`, so a few "human" pauses may not involve a person.
- **Older Codex logs are less exact.** In the 106 sessions that write each tool call together with its result, single-tool pauses match the tools' own runtimes 89% of the time, versus 98–99% elsewhere.
- **The trace is narrow.** It covers 43 developers in one lab using two closed models, so habits and context sizes may differ elsewhere.
- **Open: which model and GPU will M5 measure prefill on?**

How to reproduce

Everything rebuilds from the public data in under a minute. The commands below are for Windows PowerShell, such as the VS Code terminal; Mac and Linux differences are at the end.

1. Set up TraceLab (once)

Clone outside OneDrive or any other synced folder, since the data file is about 100 MB.

```
cd $HOME
git clone https://github.com/uw-syfi/TraceLab.git
cd TraceLab
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 |
iex"
```

Close and reopen the terminal so it finds `uv`, then run this from the TraceLab folder:

```
uv --version
uv sync
```

If `uv` is still not found, run `$env:Path = "$HOME\.local\bin;$env:Path"`, which fixes it for that terminal only.

2. Download and check the data

```
curl.exe -L --fail --ssl-no-revoke -o trace\syfi_coding_trace.duckdb
https://github.com/uw-
syfi/TraceLab/releases/download/v0.0.1/syfi_coding_trace.duckdb
uv run python artifacts\trace_facts\overview_summary\analyze.py --db
trace\syfi_coding_trace.duckdb
```

The summary should report 357,161 LLM rounds, 432,510 tool calls and 43 distinct users.

3. Add our scripts

Copy the six `.py` files from our repo's `analysis/pauses` folder into a new `our_work` folder inside TraceLab:

```
mkdir our_work
Copy-Item <path-to-our-repo>\analysis\pauses\*.py our_work\
```

4. Run the scripts from the TraceLab folder

The scripts use paths relative to the TraceLab folder, so run them from there, in this order:

```
uv run python our_work\build_pause_table.py
uv run python our_work\look_at_pauses.py
uv run python our_work\plot_pause_cdf.py
uv run python our_work\plot_pause_by_tool.py
uv run python our_work\plot_context_size.py
```

`build_pause_table.py` must run first, because the other four read the `pauses.parquet` file it writes. The three plot scripts save `fig_pause_cdf.png`, `fig_pause_by_tool.png` and `fig_context_size.png` in `our_work`. To view any one session's raw events, run `uv run python our_work\look_at_session.py <session_id>`.

5. Check the results

Check	Expected
Rows in `pauses.parquet`	357,161, of which 351,497 usable
Claude tool pauses (build_pause_table)	median 0.12 s, 90th percentile 14.3 s
Codex tool pauses (build_pause_table)	median 0.33 s, 90th percentile 10.0 s
Check session (look_at_pauses)	0.043, 4.56, 3.285, 5.809, 37.615, 40.533, 1.206, 94.702 and 42.993 s, then end
Shorter than b = 10 s (plot_pause_cdf)	tool 89.2%, human 11.9%, all 81.1%
Keep/evict check (plot_pause_by_tool)	89.3% always keep, 89.8% by tool name, 6.4% of long pauses caught
Prefill at 10,000 tokens/s (plot_context_size)	median b 12.5 s; keep wins 89.4% tool, 16.3% human, 81.7% all

The keep/evict check picks its test sessions with an MD5 hash of the session ID, so its numbers come out the same on every machine.

Mac and Linux

- Install uv with `curl -LsSf https://astral.sh/uv/install.sh | sh`.
- Use `curl` instead of `curl.exe`, and drop `--ssl-no-revoke`.

- Use `/` instead of `\` in paths, and `cp <path-to-our-repo>/analysis/pauses/*.py our_work/` to copy the scripts.